You can obtain the `std::future` object associated with a given `std::promise` by calling the `get_future()` member function, just like with `std::packaged_task`. When the value of the promise is set (using the `set_value()` member function), the future becomes *ready* and can be used to retrieve the stored value. If you destroy the `std::promise` without setting a value, an exception is stored instead. Section 4.2.4 describes how exceptions are transferred across threads.

Listing 4.10 shows some example code for a thread processing connections as just described. In this example, you use a `std::promise<bool>`/`std::future<bool>` pair to identify the successful transmission of a block of outgoing data; the value associated with the future is a simple success/failure flag. For incoming packets, the data associated with the future is the payload of the data packet.

Listing 4.10. Handling multiple connections from a single thread using promises

```
#include <future>

void process_connections(connection_set& connections)
{
    while(!done(connections))    <-(1)
    {
        for(connection_iterator       <-(2)
                connection=connections.begin(),end=connections.end();
            connection!=end;
            ++connection)
        {
            if(connection->has_incoming_data())    <-(3)
            {
                data_packet data=connection->incoming();
                std::promise<payload_type>& p=
                    connection->get_promise(data.id);    <-(4)
                p.set_value(data.payload);
            }
            if(connection->has_outgoing_data())    <-(5)
            {
                outgoing_packet data=
                    connection->top_of_outgoing_queue();
                connection->send(data.payload);
                data.promise.set_value(true);      <-(6)
            }
        }
    }
}
```

The function `process_connections()` loops until `done()` returns `true` (1). Every time through the loop, it checks each connection in turn (2), retrieving incoming data if there is any (3) or sending any queued outgoing data (5). This assumes that an incoming packet has some ID and a payload with the actual data in it. The ID is mapped to a `std::promise` (perhaps by a lookup in an associative container) (4), and the value is set to the packet's payload. For outgoing packets, the packet is retrieved from the outgoing queue and actually sent through the connection. Once the send has completed, the promise associated with the outgoing data is set to `true` to indicate successful transmission (6). Whether this maps nicely to the actual network protocol depends on the protocol; this promise/future style structure may not work for a particular scenario, although it does have a similar structure to the asynchronous I/O support of some operating systems.